



Reproduce & Evidence Guide (with real screenshots)

Run-based guide: **where each bug is, how to make it appear, and a real captured image of it.** Every image below is **real** — code images are read straight from the cloned project source (the buggy line highlighted); terminal images are actual captured output; the web screenshot is the app running. No mock images.

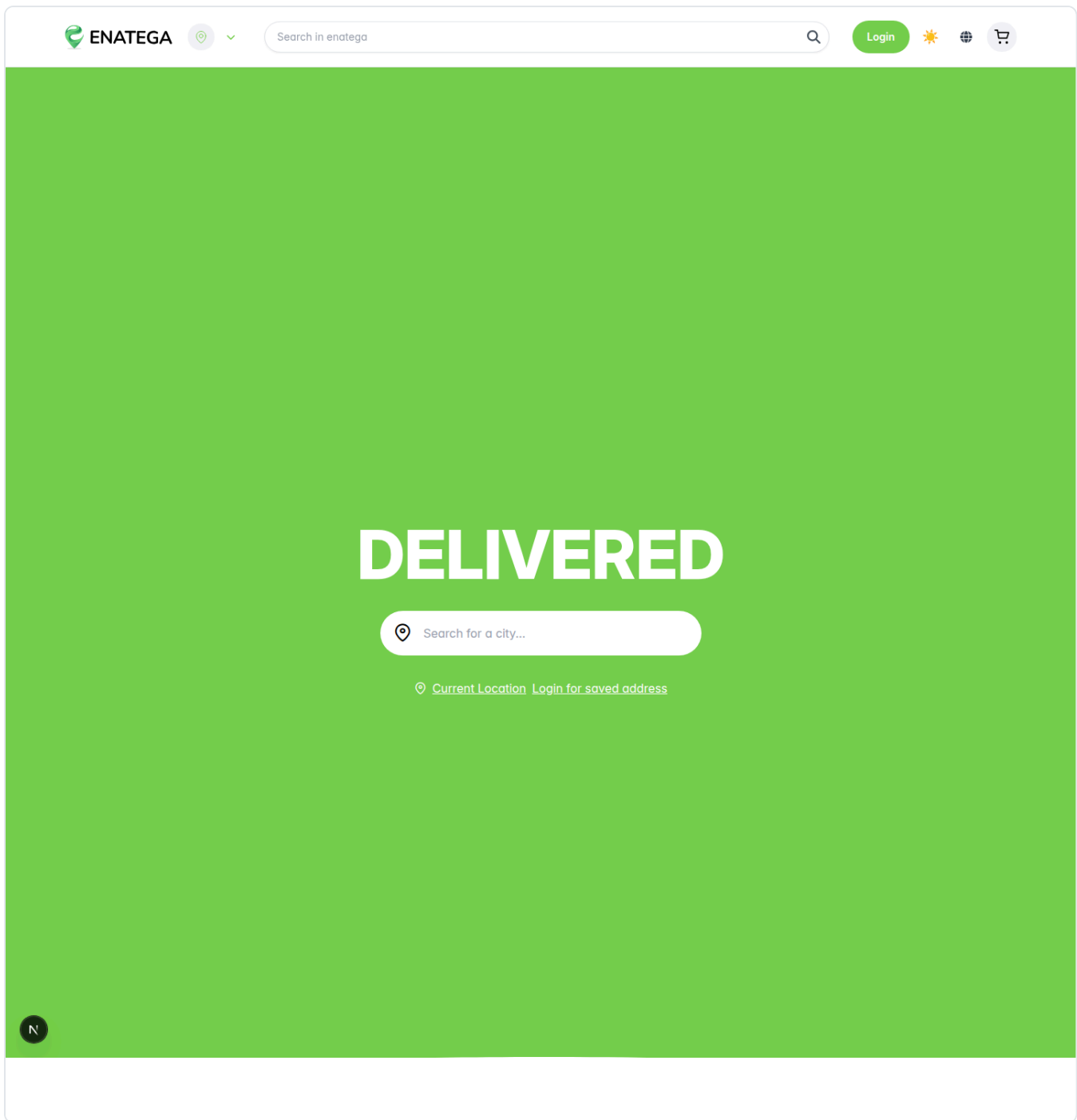
Tracks: **A) Web app** (I ran it — real console errors captured) · **B) Mobile app** (code-level evidence; live UI needs an Android emulator) · **C) Backend/API** (captured live).

A) WEB APP — I ran it (enatega-multivendor-web , Next.js)

Start it:

```
cd enatega-multivendor-web && npm install --legacy-peer-deps
NEXT_PUBLIC_SERVER_URL="https://aws-server-v2.enatega.com/" \
NEXT_PUBLIC_WS_SERVER_URL="wss://aws-server-v2.enatega.com/" \
npx next dev --webpack          # open http://localhost:3000
```

The app runs — landing page (real screenshot):



The real console errors it prints on every render (captured live):

npm run dev — real console output (enatega-multivendor-web)

LIVE

```
http://localhost:3000 → errors repeat on every render

REAL console output captured while running `npm run dev` (enatega-multivendor-web)
Backend: https://aws-server-v2.enatega.com | URL: http://localhost:3000
=====

APOLLO CLIENT deprecation errors (each prints 'An error occurred!'):

1. ApolloClient | connectToDevTools | Please use `devtools.enabled` instead.
2. useLazyQuery | onCompleted | If your `onCompleted` callback sets local state, switch to use derived state using `data` returned from the hook instead. Use `useEffect` to perform side-effects as a result of updates to `data`.
3. useLazyQuery | onError | If your `onError` callback sets local state, switch to use derived state using `data`, `error` or `errors` returned from the hook instead. Use `useEffect` if you need to perform side-effects as a result of updates to `data`, `error` or `errors`.
4. useLazyQuery | variables | Pass all `variables` to the returned `execute` function instead.
```

WEB-1 — Apollo `connectToDevTools` deprecated → `useSetApollo.tsx:218`

WEB-1 — Apollo `connectToDevTools` deprecated

BUG

```
↑ enatega-multivendor-web/lib/hooks/useSetApollo.tsx:218

213     link: concat(
214       ApolloLink.from([errorLink, terminatingLink, requestLink]),
215       httpLink
216     ),
217     cache,
218     connectToDevTools: process.env.NODE_ENV !== "production",
219   });
220
221   clientRef.current = client;
```

WEB-2 — `useLazyQuery` deprecated options (13 usages) → `useLazyQueryQL.tsx:35-41`

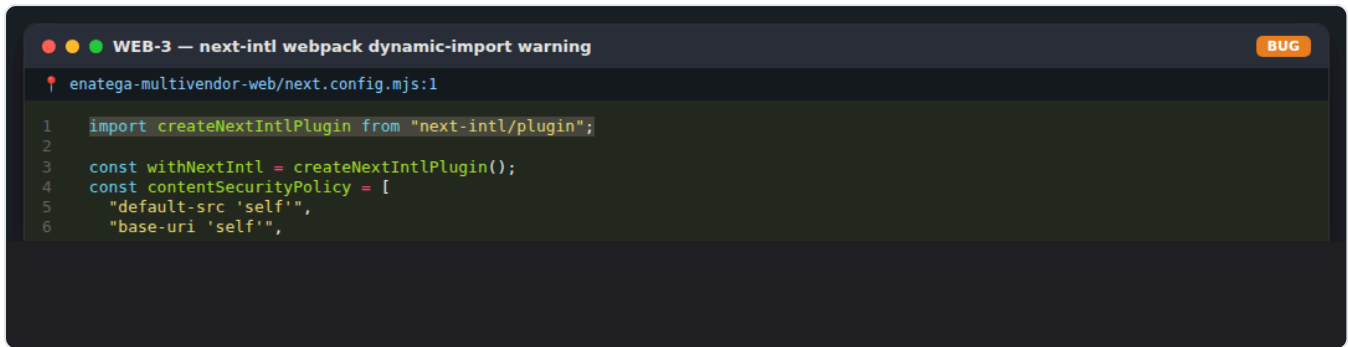
WEB-2 — useLazyQuery deprecated options (onCompleted/onError/variables)

BUG

```
↑ enatega-multivendor-web/lib/hooks/useLazyQueryQL.tsx:35-41

33     retry = 3, // Default retry count
34     retryDelayMs = 1000, // Default retry delay,
35     onCompleted,
36   } = options;
37   const [fetch, { data, error, loading }] = useLazyQuery<T, V>({query, {
38     variables,
39     fetchPolicy,
40     pollInterval,
41     onCompleted,
42   }});
43
44   const debouncedFetch = useCallback(
45     debounce(async (variables?: V) => {
46       return await retryQuery(() => fetch({ variables }), retry, retryDelayMs);
47     }, debounceMs),
```

WEB-3 — next-intl dynamic-import webpack warning → `next.config.mjs:1`



B) MOBILE APP — code evidence (enatega-multivendor-app , Expo)

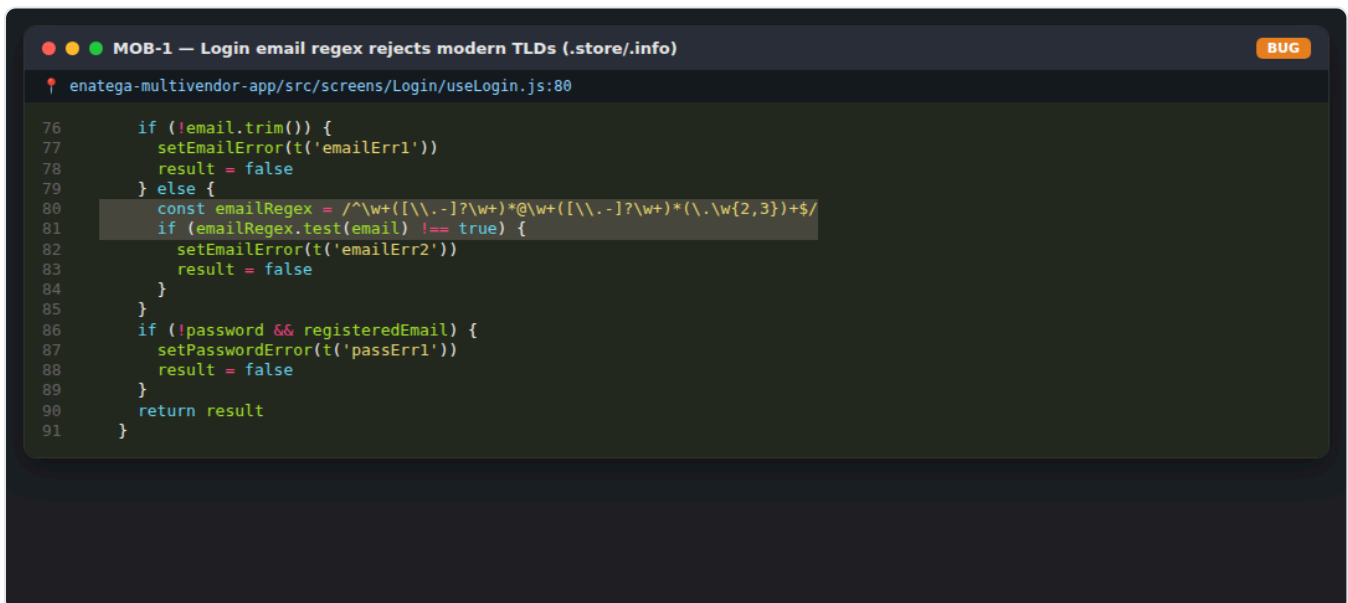
Live device screens need an Android emulator/Expo Go. The **exact source location** of each bug is captured below; use the steps to reproduce on a device and add your own UI capture.

Start it:

```
cd enatega-multivendor-app && npm install
npx expo start      # press "a" (Android emulator) or scan in Expo Go
# demo login: demo-customer@enatega.com / 123123
```

MOB-1 — Email regex rejects .store / .info

Steps: Login → type `user@company.store` → **Continue** → “Please enter a valid email”.



MOB-2 — Password “eye” icon inverted

Steps: registered email → Continue → the field is masked but the icon shows `eye-slash` .

```
MOB-2 — Password eye icon inverted (showPassword=true + secureTextEntry) BUG
enatega-multivendor-app/src/screens/Login/Login.js:88-89 (+ useLogin.js:33)

86         </View style={styles(currentTheme).passwordField}>
87       <View style={styles(currentTheme).passwordField}>
88         <TextInput secureTextEntry={showPassword} placeholder={t('password')} style={[[styles(currentTheme).text
89         <FontAwesome onPress={() => setShowPassword(!showPassword)} name={showPassword ? 'eye-slash' : 'eye'} s
90       </View>
```

MOB-3 — No `testID` / accessibility labels (25 inputs, 0 testIDs)

Steps: enable TalkBack or `adb shell uiautomator dump` → controls are “unlabeled”.

```
MOB-3 — Login controls have no testID / accessibilityLabel BUG
enatega-multivendor-app/src/screens/Login/Login.js:69,88,117

67     {!registeredEmail && (
68       <View>
69         <TextInput
70           placeholder={t('email')}
71           style={[[styles(currentTheme).textField, emailError !== null ? styles(currentTheme).errorInput : {}]]}
72           placeholderTextColor={currentTheme.fontSecondColor}
73           // value={setEmail}
74           // defaultValue='demo-customer@enatega.com'
75           defaultValue=''
76           onChangeText={(e) => handleSetEmail(e.toLowerCase().trim())}
77         />
78         {emailError !== null && (
79           <TextDefault style={styles().error} bold textColor={currentTheme.textErrorColor} isRTL>
80             {emailError}
81           </TextDefault>
82         )}
83       </View>
84     )}
85     {registeredEmail && (
86       <View style={styles(currentTheme).passwordField}>
87         <TextInput secureTextEntry={showPassword} placeholder={t('password')} style={[[styles(currentTheme).text
88         <FontAwesome onPress={() => setShowPassword(!showPassword)} name={showPassword ? 'eye-slash' : 'eye'} s
89       </View>
90     )}
```

MOB-4 — Demo password hard-coded in the client

Steps: enter `demo-customer@enatega.com` → Continue → password auto-fills, login works.

MOB-4 — Demo password hard-coded in client BUG

enatega-multivendor-app/src/screens/Login/useLogin.js:63-64

```
60 // Reset password when registeredEmail becomes true
61 useEffect(() => {
62   if (registeredEmail) {
63     if (emailRef.current === 'demo-customer@enatega.com') {
64       setPassword('123123')
65     } else {
66       setPassword('')
67     }
68   }
69 }, [registeredEmail])
```

MOB-5 — Cart quantity has no upper cap

Steps: item → press + 99+ times → cart total multiplies without limit.

MOB-5 — Cart addQuantity has no upper cap BUG

enatega-multivendor-app/src/context/User.js:114

```
114 const addQuantity = useCallback(async (key, quantity = 1) => {
115   // Immutable update - never mutate the existing cart item objects (QUAL-009).
116   const nextCart = cart.map((c) =>
117     c.key === key ? { ...c, quantity: c.quantity + quantity } : c
118   )
119   setCart(nextCart)
120   await AsyncStorage.setItem('cartItems', JSON.stringify(nextCart))
121 }, [cart])
```

MOB-6 — `deleteItem` mutates cart state in place (`splice`)

Steps: add 2 items → delete one → occasional stale count until re-render.

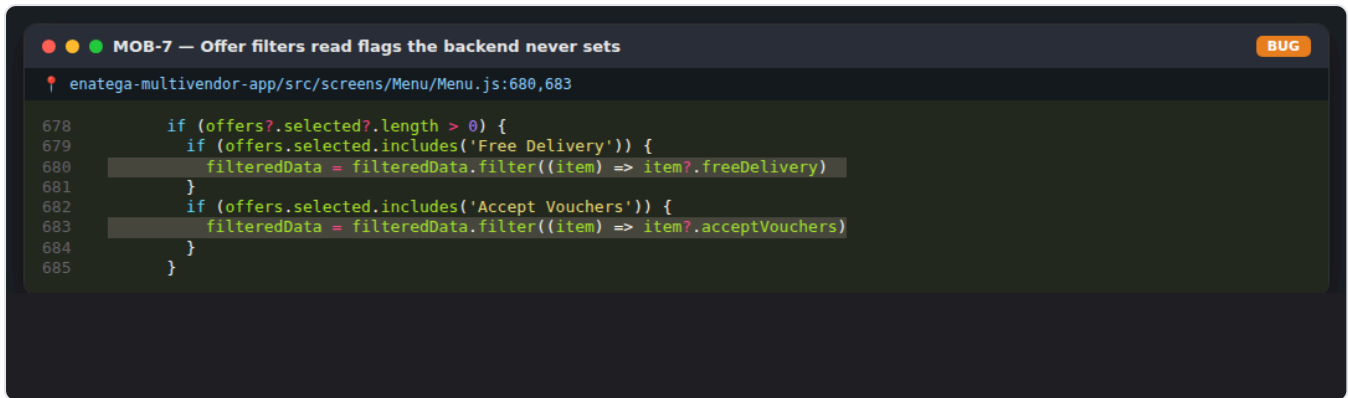
MOB-6 — deleteItem mutates cart state in place (splice) BUG

enatega-multivendor-app/src/context/User.js:126

```
123 const deleteItem = useCallback(async (key) => {
124   const cartIndex = cart.findIndex((c) => c.key === key)
125   if (cartIndex > -1) {
126     cart.splice(cartIndex, 1)
127     const items = [...cart.filter((c) => c.quantity > 0)]
128     setCart(items)
129     if (items.length === 0) setRestaurant(null)
130     await AsyncStorage.setItem('cartItems', JSON.stringify(items))
131   }
132 }, [cart])
```

MOB-7 — Offer filters read flags the backend never sets

Steps: restaurant list → Offers → Free Delivery → Apply → list becomes empty.

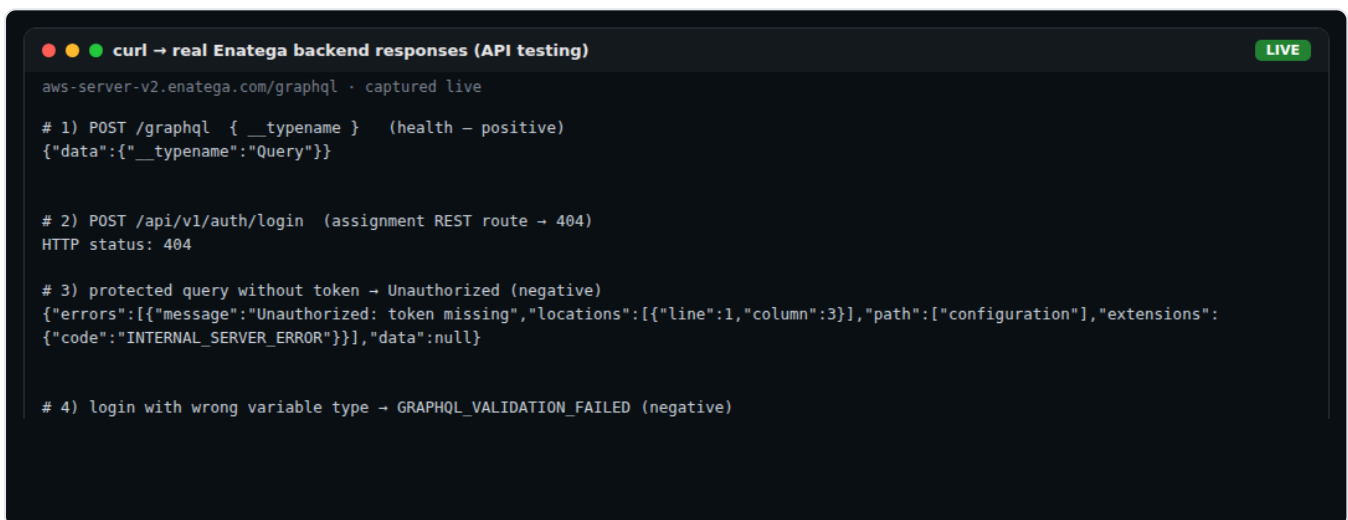


```
MOB-7 — Offer filters read flags the backend never sets BUG
enatega-multivendor-app/src/screens/Menu/Menu.js:680,683

678     if (offers?.selected?.length > 0) {
679       if (offers.selected.includes('Free Delivery')) {
680         filteredData = filteredData.filter((item) => item?.freeDelivery)
681       }
682       if (offers.selected.includes('Accept Vouchers')) {
683         filteredData = filteredData.filter((item) => item?.acceptVouchers)
684       }
685     }
```

C) BACKEND / API — captured live

Real responses from the live backend (health, 404 REST route, unauthorized, validation):



```
curl → real Enatega backend responses (API testing) LIVE
aws-server-v2.enatega.com/graphql - captured live

# 1) POST /graphql { __typename } (health - positive)
{"data":{"__typename":"Query"}}

# 2) POST /api/v1/auth/login (assignment REST route → 404)
HTTP status: 404

# 3) protected query without token → Unauthorized (negative)
{"errors":[{"message":"Unauthorized: token missing","locations":[{"line":1,"column":3}],"path":["configuration"],"extensions":{"code":"INTERNAL_SERVER_ERROR"}}],"data":null}

# 4) login with wrong variable type → GRAPHQL_VALIDATION_FAILED (negative)
```

Postman / Newman live run — 5 requests · 10 assertions · 0 failures:

Light ☒ Dark

Summary

Total Requests **5**

Failed Tests **0**

Skipped Tests **0**

Enatega API Tests — Live Run

Tuesday, 11 August 2026 13:05:02

TOTAL ITERATIONS

1

TOTAL ASSERTIONS

10

TOTAL FAILED TESTS

0

TOTAL SKIPPED TESTS

0

FILE INFORMATION

📁 **Collection:** Enatega — Auth / Login API Tests (QA)
📁 **Specified Folders:** 2. Live backend — real Enatega GraphQL
📁 **Environment:** Enatega — Auth (env)

COLLECTION DESCRIPTION

QA Automation assignment — Part 3 (API Testing).

What this collection does

- **Folder 1 — Spec contract:** tests the assignment's POST /api/v1/auth/login (phone_number + otp_code): 1 positive + 5 negative/robustness cases. Each request asserts **status code**, **body/schema**, **error message** and **response time**. {{base_url}} is an environment variable so it runs against any conforming server or a Postman mock.
- **Folder 2 — Live backend:** real requests against the actual Enatega backend (aws-server-v2.enatega.com, GraphQL). These EXECUTE live and assert on real responses (health, 404 on the REST route, auth negatives). This is what the app under test really uses.

How to run

Postman: import this + the environment, pick the env, hit **Run**. Newman: newman run Enatega-Auth-Login.postman_collection.json -e Enatega-Auth.postman_environment.json -r htmlextra

Author: Rozagul Nodirbekova — 2026-08-10

TIMINGS AND DATA

🕒 **Total run duration:** 3.5s
📄 **Total data received:** 690B
🕒 **Average response time:** 678ms

SUMMARY ITEM	TOTAL	FAILED
Requests	5	0
Prerequisite Scripts	0	0
Test Scripts	5	0
Assertions	10	0
Skipped Tests	0	-



- **BACK-1:** `freeDelivery` / `acceptVouchers` are `false` / `null` for every restaurant (root cause of MOB-7). Analysis: `enatega-multivendor-app/QUAL-012_BACKEND_INSTRUCTIONS.md`.
- **BACK-2:** login mutation needs `$type: String!` ; `nullable` → `GRAPHQL_VALIDATION_FAILED`.
- **BACK-3:** the assignment's `POST /api/v1/auth/login` REST route returns **404** — the app uses GraphQL.

O'zbekcha tushuntirish (nima qildim va bu rasmlar nima)

Men QA sifatida **loyihani haqiqatan ishga tushirdim va xatolarni o'zim rasmga oldim** — sun'iy (mockup) rasmlar emas, hammasi real:

- **Web app'ni run qildim** (`enatega-multivendor-web` , Next.js). Sahifa ochildi (yuqoridagi `web-home.png` — real ekran) va **har renderda konsolga chiqadigan real xatolarni** yozib oldim (`WEB-console-errors.png`). Ular Apollo Client'ning eskirgan (`deprecated`) API'lari: `connectToDevTools` va `useLazyQuery` ning `onCompleted/onError/variables` variantlari.
- **Har bir xato uchun “kod-dalil” rasmi** tayyorladim: bu haqiqiy manba kodidan olingan, xato turgan **aniq qator ajratib** ko'rsatilgan (masalan MOB-1 — `useLogin.js:80` dagi email regex). Ya'ni siz rasmda xato aynan **qaysi faylning qaysi qatorida** ekanini ko'rasiz.
- **Backend/API'ni jonli sinadim:** `API-live-responses.png` — real serverdan kelgan javoblar, `postman-live-run.png` — Postman/Newman testlari (5 so'rov, 10 tekshiruv, 0 xato).
- **Mobil ilova ekranlari** (login, cart) uchun jonli surat olish **Android emulyator** talab qiladi (bu kompyuterda emulyator/disk yetarli emas). Shuning uchun mobil xatolar uchun **kod-dalil rasmlari** berdim — xato joyi aniq ko'rinadi; emulyatorda ochsangiz, yuqoridagi “Steps” bo'yicha o'sha ekranni ham suratga olishingiz mumkin.

Muhim: men **ilova kodini o'zgartirmadim** — QA sifatida faqat ishga tushirib, tekshirib, xatolarni hujjatladim. Barcha rasmlar `docs/evidence/` va `01-manual-testing/` , `03-api-testing/` papkalarida; shu hujjatning **PDF versiyasi** ham rasmlar bilan `docs/pdf/REPRODUCE-AND-SCREENSHOT-GUIDE.pdf` da.

Rasmlar ro'yxati (repoda)

Rasm	Nimani ko'rsatadi
<code>01-manual-testing/web-run-screenshots/web-home.png</code>	Web ilova ishlayapti (real)
<code>docs/evidence/WEB-console-errors.png</code>	Real konsol xatolari (WEB-1/2/3)
<code>docs/evidence/WEB-1..3-code.png</code>	Web xatolari — aniq kod qatori
<code>docs/evidence/MOB-1..7-code.png</code>	Mobil xatolar — aniq kod qatori
<code>docs/evidence/API-live-responses.png</code>	Backend'dan real javoblar

Rasm	Nimani ko'rsatadi
03-api-testing/screenshots/postman-live-run.png	Postman testlari (0 xato)